

Micro-Expression 3D CNN - Line-by-Line Explanation with Improvements

Section 1: Basic Dataset and Model

Lines 1-5: Imports

```
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
from pathlib import Path
```

What they do:

- torch: Core PyTorch library for tensor operations and neural networks
- Dataset, DataLoader: Base classes for creating custom datasets and loading data in batches
- transforms: Image preprocessing functions (resize, normalize, etc.)
- Image: PIL library for loading and manipulating images
- Path: Modern way to handle file paths (better than os.path)

Improvements:

- Add typing imports for better code documentation: `from typing import Optional, Tuple, List`
- Add numpy import as it's commonly needed: `import numpy as np`

Lines 7-10: Class Definition

```
class MicroExp3DDataset(Dataset):
    def __init__(self, root_dir, transform=None):
```

What they do:

- Defines custom dataset class inheriting from PyTorch's Dataset
- `__init__` takes root directory path and optional transform

Improvements:

- Add type hints: `def __init__(self, root_dir: str, transform: Optional[transforms.Compose] = None)`
- Add docstring explaining the expected directory structure

Lines 11-16: Docstring and Initialization

```
    """
    root_dir: path to split folder (e.g. train/, val/, test/)
    transform: torchvision transforms for each frame
    """
    self.root_dir = Path(root_dir)
    self.samples = []
    self.labels = []
```

What they do:

- Docstring documents parameters
- Converts `root_dir` to Path object for easier path manipulation
- Initializes empty lists to store sample paths and labels

Improvements:

- Add validation: `if not self.root_dir.exists(): raise FileNotFoundError(f"{root_dir} not found")`
- Use more descriptive name: `self.sample_paths` instead of `self.samples`

Lines 17-18: Build Class Mappings

```
self.classes = sorted([d.name for d in self.root_dir.iterdir() if d.is_dir()])
self.class_to_idx = {cls: i for i, cls in enumerate(self.classes)}
```

What they do:

- Gets all subdirectory names (class names) and sorts them alphabetically
- Creates dictionary mapping class names to integer indices (0, 1, 2, etc.)

Improvements:

- Add check for empty directories: `if not self.classes: raise ValueError("No class directories found")`
- Print class information: `print(f"Found {len(self.classes)} classes: {self.classes}")`

Lines 20-24: Load Sample Paths

```
for cls in self.classes:
    for sample_dir in (self.root_dir / cls).iterdir():
        if sample_dir.is_dir():
            self.samples.append(sample_dir)
            self.labels.append(self.class_to_idx[cls])
```

What they do:

- Iterates through each class folder
- For each class, finds all subdirectories (each represents one video sample)
- Appends the sample path and its corresponding label

Improvements:

- Sort samples for reproducibility: `for sample_dir in sorted((self.root_dir / cls).iterdir())`
- Add validation: Skip empty directories or log warnings
- Use tuple of lists or numpy arrays for better memory efficiency with large datasets

Lines 26-29: Default Transform

```
self.transform = transform or transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor()
])
```

What they do:

- If no transform provided, uses default: resize to 128x128 and convert to tensor
- `ToTensor()` converts PIL Image to torch tensor and scales from [0,255] to [0,1]

Improvements:

- Add normalization with ImageNet stats: `transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`
- Consider making image size configurable: `Resize((self.img_size, self.img_size))`

Lines 31-32: Dataset Length

```
def __len__(self):
    return len(self.samples)
```

What they do:

- Required method for PyTorch Dataset
- Returns total number of samples

Improvements:

- None needed, this is correct

Lines 34-35: Get Item Method

```
def __getitem__(self, idx):
    sample_dir = self.samples[idx]
    label = self.labels[idx]
```

What they do:

- Required method that returns one sample and its label
- Gets the directory path and label for the given index

Improvements:

- Add bounds checking: `if idx >= len(self.samples): raise IndexError`
- Consider caching for faster repeated access

Lines 37-42: Load Frames

```
frame_paths = sorted(sample_dir.glob("*.jpg"))
frames = []
for frame_path in frame_paths:
    img = Image.open(frame_path).convert('RGB')
    img = self.transform(img)
    frames.append(img)
```

What they do:

- Finds all JPG files in the sample directory and sorts them
- Iterates through each frame
- Opens image, converts to RGB (handles grayscale or RGBA)
- Applies transforms (resize, convert to tensor)
- Appends to frames list

Improvements:

- Handle variable frame counts: Add padding or sampling to fixed length
- Add error handling: `try-except` for corrupt images
- Use list comprehension for efficiency: `frames = [self.transform(Image.open(fp).convert('RGB')) for fp in frame_paths]`
- Check if frames list is empty before stacking

Lines 44-45: Stack and Return

```
video_tensor = torch.stack(frames, dim=1)
return video_tensor, label
```

What they do:

- Stacks list of frame tensors along dimension 1 (time dimension)
- Returns shape: (C, T, H, W) where C=channels, T=time/frames, H=height, W=width
- Returns tuple of (video_tensor, label)

Improvements:

- Add assertion to verify shape: `assert video_tensor.dim() == 4, f"Expected 4D tensor, got {video_tensor.dim()}D"`
- Consider returning dictionary for clarity: `return {'video': video_tensor, 'label': label}`

Lines 48-52: Create Transforms

```
frame_transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
])
```

What they do:

- Creates transform pipeline to be applied to each frame
- Resize ensures consistent input size
- ToTensor converts to PyTorch tensor [0,1]

Improvements:

- Add data augmentation for training: RandomHorizontalFlip, ColorJitter

- Add normalization with dataset statistics
- Create separate transforms for train/val/test

Lines 54-59: Create Datasets

```
root = "/kaggle/input/microexp/processed_32_subject_split"

train_dataset = MicroExp3DDataset(f"{root}/train", transform=frame_transform)
val_dataset = MicroExp3DDataset(f"{root}/val", transform=frame_transform)
test_dataset = MicroExp3DDataset(f"{root}/test", transform=frame_transform)
```

What they do:

- Sets root directory (hardcoded Kaggle path)
- Creates three dataset instances for train, validation, and test splits

Improvements:

- Use argparse or config file for paths instead of hardcoding
- Add validation: Check if paths exist before creating datasets
- Pass classes from train_dataset to val and test to ensure consistency
- Use Path object: `root = Path("/kaggle/input/microexp/processed_32_subject_split")`

Lines 61-63: Create DataLoaders

```
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=8, shuffle=False, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False, num_workers=2)
```

What they do:

- Creates DataLoader objects that handle batching and parallel loading
- `batch_size=8`: Loads 8 videos at a time
- `shuffle=True` for training (randomize order), `False` for val/test
- `num_workers`: Number of parallel processes for loading data

Improvements:

- Add `pin_memory=True` for faster GPU transfer
- Add `drop_last=True` for training to avoid incomplete batches
- Make `batch_size` configurable
- Consider `persistent_workers=True` for faster epoch transitions
- Reduce `num_workers` if running out of memory

Lines 65-67: Test DataLoader

```
x, y = next(iter(train_loader))
print(x.shape) # (B, C, T, H, W)
print(y.shape) # (B,)
```

What they do:

- Gets one batch from train_loader
- Prints shape of videos (x) and labels (y)
- Verifies data pipeline is working

Improvements:

- Add more detailed output: `print(f"Video shape: {x.shape}, dtype: {x.dtype}, range: [{x.min():.2f}, {x.max():.2f}])")`
- Print label distribution: `print(f"Labels in batch: {y.tolist()}")`
- Visualize a sample frame to ensure preprocessing is correct

Lines 69-72: Device Selection

```
if torch.cuda.is_available():
    device = "cuda"
else:
    device = "cpu"
```

What they do:

- Checks if CUDA-capable GPU is available
- Sets device to "cuda" if available, otherwise "cpu"

Improvements:

- Use torch.device: `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`
 - Print device info: `print(f"Using device: {device}")`
 - Check multiple GPUs: `torch.cuda.device_count()`
 - Set specific GPU: `torch.cuda.set_device(0)`
-

Lines 74-76: Model Imports

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

What they do:

- Import neural network module (nn) for layers
- Import functional API (F) for activation functions and operations

Improvements:

- These imports should be at the top of the file
 - Remove duplicate `import torch` (already imported at line 1)
-

Lines 78-80: Model Class Definition

```
class Simple3DCNN(nn.Module):
    def __init__(self, num_classes=3):
        super(Simple3DCNN, self).__init__()
```

What they do:

- Defines 3D CNN model inheriting from nn.Module
- Takes num_classes parameter (default 3 for 3 emotions)
- Calls parent class constructor

Improvements:

- Add type hints: `def __init__(self, num_classes: int = 3, dropout_p: float = 0.5)`
 - Add docstring explaining architecture
 - Consider making more parameters configurable (channels, kernel sizes)
-

Lines 82-94: Feature Extraction Layers

```

self.features = nn.Sequential(
    nn.Conv3d(3, 32, kernel_size=(3, 3, 3), stride=1, padding=1),
    nn.BatchNorm3d(32),
    nn.ReLU(inplace=True),
    nn.MaxPool3d((1, 2, 2)),

    nn.Conv3d(32, 64, kernel_size=(3, 3, 3), stride=1, padding=1),
    nn.BatchNorm3d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool3d((2, 2, 2)),

    nn.Conv3d(64, 128, kernel_size=(3, 3, 3), stride=1, padding=1),
    nn.BatchNorm3d(128),
    nn.ReLU(inplace=True),
    nn.MaxPool3d((2, 2, 2)),

```

What they do: Block 1:

- Conv3d(3, 32): Takes 3 input channels (RGB), outputs 32 feature maps
- kernel_size=(3,3,3): 3x3x3 filter (temporal + spatial)
- BatchNorm3d: Normalizes activations for stable training
- ReLU: Non-linear activation, inplace=True saves memory
- MaxPool3d(1,2,2): Pools spatial dims by 2, keeps temporal unchanged

Block 2:

- Conv3d(32, 64): Doubles channels to 64
- MaxPool3d(2,2,2): Reduces all dimensions by 2

Block 3:

- Conv3d(64, 128): Doubles channels to 128
- MaxPool3d(2,2,2): Further reduces all dimensions

Improvements:

- Add dropout: `nn.Dropout(0.3)` after each block to prevent overfitting
- Use padding='same' for clearer intent (PyTorch 1.9+)
- Consider residual connections for deeper networks
- Add comments explaining output shapes at each stage
- First pooling (1,2,2) preserves temporal info - good choice!

Lines 96-101: Classifier

```

self.classifier = nn.Sequential(
    nn.AdaptiveAvgPool3d((1, 1, 1)),
    nn.Flatten(),
    nn.Linear(128, num_classes)
)

```

What they do:

- AdaptiveAvgPool3d(1,1,1): Reduces any input size to 1x1x1 per channel
- Flatten: Converts (B, 128, 1, 1, 1) to (B, 128)
- Linear(128, num_classes): Final classification layer

Improvements:

- Add dropout before Linear: `nn.Dropout(0.5)`
- Add intermediate layer: `nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.3), nn.Linear(64, num_classes)`
- Consider using global average AND max pooling combined
- Initialize weights explicitly for better convergence

Lines 103-106: Forward Pass

```

def forward(self, x):
    x = self.features(x)
    x = self.classifier(x)
    return x

```

What they do:

- Defines forward pass through the network
- Passes input through feature extraction then classifier
- Returns logits (raw scores, not probabilities)

Improvements:

- Add input shape validation: `assert x.dim() == 5, f"Expected 5D input (B,C,T,H,W), got {x.shape}"`
- Consider returning intermediate features for visualization
- Add comments showing tensor shapes at each stage

Lines 108-110: Model Instantiation

```
model = Simple3DCNN(num_classes=3).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

What they do:

- Creates model instance with 3 output classes and moves to GPU/CPU
- Creates CrossEntropyLoss (combines LogSoftmax + NLLoss)
- Creates Adam optimizer with learning rate 0.0001

Improvements:

- Add weight_decay: `optimizer = torch.optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-5)`
- Print model summary: `print(model)` or use `torchsummary`
- Count parameters: `sum(p.numel() for p in model.parameters())`
- Consider using AdamW instead of Adam for better regularization
- Use learning rate scheduler

Lines 112-118: Sanity Check

```
videos, labels = next(iter(train_loader))
videos, labels = videos.to(device), labels.to(device)
outputs = model(videos)
loss = criterion(outputs, labels)
print("Output shape:", outputs.shape) # (B, 3)
```

What they do:

- Gets one batch of data
- Moves to device (GPU/CPU)
- Runs forward pass
- Computes loss
- Prints output shape to verify model works

Improvements:

- Add gradient check: Run backward pass to ensure it works
- Print loss value: `print(f"Initial loss: {loss.item():.4f}")`
- Check for NaN: `assert not torch.isnan(loss), "Loss is NaN!"`
- Print model output range: `print(f"Output range: [{outputs.min():.2f}, {outputs.max():.2f}]"`

Section 2: Training Functions

Lines 120-122: Additional Imports

```
import torch
from tqdm import tqdm
```

What they do:

- Imports torch (duplicate, unnecessary)
- Imports tqdm for progress bars

Improvements:

- Remove duplicate import
 - Import at top of file with other imports
-

Lines 124: Device Declaration

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

What they do:

- Creates device object (better than string "cuda")

Improvements:

- This is good! Duplicate of earlier code - use once globally
-

Lines 126-128: Train Function Definition

```
def train_one_epoch(model, loader, criterion, optimizer, scaler):  
    model.train()  
    running_loss, correct, total = 0.0, 0, 0
```

What they do:

- Defines training function for one epoch
- `model.train()`: Enables dropout and batch norm training mode
- Initializes metrics: running loss, correct predictions, total samples

Improvements:

- Add type hints: `def train_one_epoch(model: nn.Module, loader: DataLoader, criterion, optimizer, scaler) -> Tuple[float, float]`
 - Add device parameter: Pass device explicitly instead of using global
 - Return both loss and accuracy for better tracking
-

Lines 130-143: Training Loop

```
for videos, labels in tqdm(loader, desc="Training", leave=False):  
    videos, labels = videos.to(device), labels.to(device)  
    optimizer.zero_grad()  
  
    with torch.cuda.amp.autocast():  
        outputs = model(videos)  
        loss = criterion(outputs, labels)  
  
    scaler.scale(loss).backward()  
    scaler.step(optimizer)  
    scaler.update()  
  
    running_loss += loss.item() * videos.size(0)  
    _, preds = torch.max(outputs, 1)  
    correct += (preds == labels).sum().item()  
    total += labels.size(0)
```

What they do:

- Iterates through all batches with progress bar
- Moves data to device
- Zeros gradients (prevents accumulation from previous batch)
- `autocast()`: Uses mixed precision (float16) for faster training
- Forward pass and loss computation
- `scaler.scale()`: Scales loss to prevent underflow with float16
- Backward pass computes gradients
- `scaler.step()`: Unscales gradients and updates weights
- `scaler.update()`: Updates scale factor for next iteration
- Accumulates loss (weighted by batch size)
- Gets predictions (argmax of logits)
- Counts correct predictions

Improvements:

- Add gradient clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` before `optimizer.step()`
- Use `non_blocking=True`: `videos.to(device, non_blocking=True)` for async transfer
- Add error handling: Try-except for NaN loss
- Log learning rate: `current_lr = optimizer.param_groups[0]['lr']`
- Consider gradient accumulation for effective larger batch sizes

Lines 145-148: Return Metrics

```
avg_loss = running_loss / total
acc = correct / total
return avg_loss, acc
```

What they do:

- Computes average loss per sample
- Computes accuracy (correct / total)
- Returns both metrics

Improvements:

- Add epsilon to prevent division by zero: `acc = correct / max(total, 1)`
- Return additional metrics: F1 score, per-class accuracy
- Consider using `torchmetrics` library for more robust metrics

Lines 151-153: Validation Function

```
def validate_one_epoch(model, loader, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
```

What they do:

- Defines validation function
- `model.eval()`: Disables dropout, sets batch norm to inference mode
- Initializes metric accumulators

Improvements:

- Add type hints
- Consider returning confusion matrix for detailed analysis
- Add device parameter

Lines 155-168: Validation Loop

```
with torch.no_grad():
    for videos, labels in tqdm(loader, desc="Validating", leave=False):
        videos, labels = videos.to(device), labels.to(device)
        outputs = model(videos)
        loss = criterion(outputs, labels)
        running_loss += loss.item() * videos.size(0)

        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

avg_loss = running_loss / total
acc = correct / total
return avg_loss, acc
```

What they do:

- `no_grad()`: Disables gradient computation (saves memory and speeds up)
- Similar to training loop but no backpropagation
- Accumulates metrics and returns

Improvements:

- Add `autocast` for validation too (faster inference)
- Store predictions and labels for confusion matrix
- Add per-class metrics
- Consider using `@torch.inference_mode()` instead of `no_grad()` (slightly faster)

Section 3: Full Training Loop

Lines 170-172: Training Configuration

```
num_epochs = 20
best_val_acc = 0.0
```

What they do:

- Sets number of epochs to train
- Initializes best validation accuracy tracker

Improvements:

- Make configurable via config file or argparse
 - Add more tracking variables: `best_epoch`, `train_losses`, `val_losses`
 - Consider early stopping patience parameter
-

Lines 174-176: Model Setup

```
model = Simple3DCNN(num_classes=3).to(device)
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-5)
```

What they do:

- Creates fresh model instance
- Creates loss function
- Creates optimizer with weight decay (L2 regularization)

Improvements:

- Add class weights if dataset is imbalanced: `criterion = nn.CrossEntropyLoss(weight=class_weights)`
 - Add label smoothing: `nn.CrossEntropyLoss(label_smoothing=0.1)`
 - Use AdamW: `torch.optim.AdamW(...)`
 - Add learning rate scheduler
-

Lines 178-179: GradScaler

```
scaler = torch.amp.GradScaler('cuda')
```

What they do:

- Creates gradient scaler for mixed precision training
- New API (older versions used `torch.cuda.amp.GradScaler()`)

Improvements:

- Check device type: `scaler = torch.amp.GradScaler('cuda' if device.type=='cuda' else 'cpu')`
 - Disable if not using GPU: `enabled=device.type=='cuda'`
-

Lines 181-223: Main Training Loop

```

for epoch in range(num_epochs):
    # Training
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for videos, labels in tqdm(train_loader, desc=f"Epoch {epoch+1} Training", leave=False):
        videos, labels = videos.to(device), labels.to(device)
        optimizer.zero_grad()

        with torch.amp.autocast('cuda'):
            outputs = model(videos)
            loss = criterion(outputs, labels)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        running_loss += loss.item() * videos.size(0)
        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss / total
    train_acc = correct / total

    # Validation
    model.eval()
    val_loss_sum, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad():
        for videos, labels in tqdm(val_loader, desc=f"Epoch {epoch+1} Validation", leave=False):
            videos, labels = videos.to(device), labels.to(device)
            with torch.amp.autocast(device_type='cuda'):
                outputs = model(videos)
                loss = criterion(outputs, labels)

            val_loss_sum += loss.item() * videos.size(0)
            _, preds = torch.max(outputs, 1)
            val_correct += (preds == labels).sum().item()
            val_total += labels.size(0)

    val_loss = val_loss_sum / val_total
    val_acc = val_correct / val_total

    print(f"Epoch [{epoch+1}/{num_epochs}] "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc*100:.2f}% | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc*100:.2f}%")

```

What they do:

- Main loop that trains for num_epochs iterations
- Each epoch: runs full training pass, runs full validation pass
- Prints metrics after each epoch
- Uses mixed precision for both training and validation

Improvements:

- Use the defined functions: `train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, scaler)`
- Add learning rate scheduler: `scheduler.step(val_loss)`
- Log to tensorboard/wandb for better visualization
- Add early stopping logic
- Save training history to file
- Add time tracking: `import time; start = time.time()`
- Add memory usage tracking

Lines 225-228: Model Checkpointing

```

if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save(model.state_dict(), "best_3dcnn_model.pth")
    print(f"Best model saved with Val Acc: {best_val_acc*100:.2f}%")

```

What they do:

- Checks if current validation accuracy is best so far
- Updates best_val_acc
- Saves model weights to file
- Prints confirmation

Improvements:

- Save complete checkpoint: `torch.save({'epoch': epoch, 'model': model.state_dict(), 'optimizer': optimizer.state_dict(), 'val_acc': val_acc}, 'checkpoint.pth')`
- Add timestamp to filename for versioning
- Save in separate directory: `Path('checkpoints').mkdir(exist_ok=True)`
- Keep top-k models instead of just best one
- Save optimizer and scheduler states for resuming training

Section 4: Advanced R3D-18 Model

Lines 230-243: Configuration Block

```

import os
from pathlib import Path
import random
import numpy as np
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import torchvision

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEED = 42

```

What they do:

- Imports all necessary libraries
- Sets device
- Sets random seed for reproducibility

Improvements:

- Organize imports: stdlib, third-party, local
- Add: `from typing import Dict, List, Tuple, Optional`
- Comment why SEED=42 (reproducibility)

Lines 244-247: Seed Everything

```

torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

```

What they do:

- Sets seeds for PyTorch, NumPy, and Python random
- Ensures reproducible results across runs

Improvements:

- Add CUDA seeds: `torch.cuda.manual_seed_all(SEED)`
- Add deterministic behavior: `torch.backends.cudnn.deterministic = True`
- Note: Deterministic mode may slow training

- Create `seed_everything()` function for reusability

Lines 249-260: Hyperparameters

```
DATA_ROOT = Path("/kaggle/input/microexp/processed_32_subject_split")
BATCH_SIZE = 6
NUM_WORKERS = 4
IMG_SIZE = (128,128)
FRAMES = 32
NUM_CLASSES = 3
LR = 1e-4
WEIGHT_DECAY = 1e-4
NUM_EPOCHS = 30
PRETRAINED = True
FREEZE_BACKBONE = False
MODEL_PATH = "r3d18_reg_best.pth"
```

What they do:

- Defines all hyperparameters as constants
- `BATCH_SIZE=6`: Smaller than before (R3D-18 needs more memory)
- `FRAMES=32`: Fixed frame count per video
- `PRETRAINED=True`: Use pretrained weights
- `FREEZE_BACKBONE=False`: Fine-tune entire network

Improvements:

- Use dataclass or config file (YAML/JSON) for better organization
- Add validation: `assert FRAMES > 0, "FRAMES must be positive"`
- Make paths OS-independent
- Add comments explaining each parameter's impact
- Consider using `argparse` for command-line configuration

Lines 262: Print Device

```
print("Device:", DEVICE)
```

What they do:

- Prints which device is being used

Improvements:

- Add more info: `print(f"Device: {DEVICE}, CUDA available: {torch.cuda.is_available()}, GPU count: {torch.cuda.device_count()}")`
- Print GPU name: `torch.cuda.get_device_name(0)`

Lines 264-265: Dataset Section Comment

```
# Cell 2: dataset + transforms
from PIL import Image
```

What they do:

- Comment indicating new section
- Import PIL Image (already imported at top)

Improvements:

- Remove duplicate import
- Use markdown-style headers for better organization

Lines 267-273: Training Transforms

```

frame_transform = transforms.Compose([
    transforms.Resize(IMG_SIZE),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.08, contrast=0.08, saturation=0.02),
    transforms.ToTensor(), # -> (C,H,W) in [0,1]
])

```

What they do:

- Resize to 128x128
- Random horizontal flip (50% chance) - data augmentation
- ColorJitter: Randomly adjusts brightness ($\pm 8\%$), contrast ($\pm 8\%$), saturation ($\pm 2\%$)
- ToTensor: Converts to tensor [0,1]

Improvements:

- Add normalization: `transforms.Normalize(mean=[0.43216, 0.394666, 0.37645], std=[0.22803, 0.22145, 0.216989])`
- Add RandomRotation: `transforms.RandomRotation(10)`
- Add RandomAffine for slight translations
- Consider RandAugment for automatic augmentation
- Compute mean/std from your actual dataset instead of using ImageNet stats

Lines 275-278: Validation Transforms

```

val_transform = transforms.Compose([
    transforms.Resize(IMG_SIZE),
    transforms.ToTensor(),
])

```

What they do:

- Simpler transform for validation (no augmentation)
- Only resize and convert to tensor

Improvements:

- Add same normalization as training: Should use identical normalization stats
- Create from training transform: `val_transform = transforms.Compose([transforms.Resize(IMG_SIZE), transforms.ToTensor()])`
- Must have identical normalization to training data

Lines 280-282: Dataset Class Definition

```

class FolderSequenceDataset(Dataset):
    """Each sample folder contains FRAMES jpg images. Returns (C,T,H,W), label_int"""
    def __init__(self, split_root, transform=None, classes=None):

```

What they do:

- Defines improved dataset class
- Docstring explains it returns (C,T,H,W) format
- Takes optional classes parameter for consistency across splits

Improvements:

- Add type hints: `def __init__(self, split_root: str, transform: Optional[transforms.Compose] = None, classes: Optional[List[str]] = None)`
- Add more parameters: `frame_sample_strategy`, `max_frames`, etc.

Lines 283-285: Initialize Dataset

```

self.root = Path(split_root)
assert self.root.exists(), f"{split_root} not found"
self.classes = sorted([d.name for d in self.root.iterdir() if d.is_dir()]) if classes is None else classes

```

What they do:

- Converts path to Path object
- Asserts path exists (good error handling!)
- Either discovers classes or uses provided ones

Improvements:

- Add check: `assert len(self.classes) > 0, "No classes found"`
- Log class info: `logger.info(f"Dataset initialized with {len(self.classes)} classes")`

Lines 286-293: Build Sample List

```
self.class_to_idx = {c:i for i,c in enumerate(self.classes)}
self.samples = []
for cls in self.classes:
    for sample_dir in sorted((self.root/cls).iterdir()):
        if sample_dir.is_dir():
            self.samples.append((sample_dir, self.class_to_idx[cls]))
self.transform = transform
```

What they do:

- Creates class to index mapping
- Iterates through classes and samples
- Stores tuples of (path, label)
- Stores transform

Improvements:

- Validate samples: Check if folders have frames
- Add sample counting per class for debugging
- Use `Path.glob()` for more flexibility: `(self.root/cls).glob('*/*')`
- Store as numpy arrays for faster access with large datasets

Lines 295-297: Length Method

```
def __len__(self):
    return len(self.samples)
```

What they do:

- Returns total sample count

Improvements:

- None needed, correct implementation

Lines 299-311: Load Frames Method

```
def _load_frames(self, folder):
    fps = sorted(folder.glob("*.jpg"))
    # ensure FRAMES length
    if len(fps) < FRAMES:
        if len(fps) == 0:
            raise RuntimeError(f"No frames in {folder}")
        last = fps[-1]
        fps = fps + [last] * (FRAMES - len(fps))
    elif len(fps) > FRAMES:
        idxs = np.linspace(0, len(fps)-1, FRAMES, dtype=int)
        fps = [fps[i] for i in idxs]
    return fps
```

What they do:

- Gets all JPG files sorted by name
- Handles variable frame counts:
 - If too few frames: Repeats last frame (padding)
 - If no frames: Raises error (good!)
 - If too many frames: Samples uniformly across sequence
- Returns exactly FRAMES paths

Improvements:

- Consider center cropping instead of padding: Only use middle FRAMES

- Add random sampling for training augmentation
 - Support other image formats: `glob("*.jpg") + glob("*.png")`
 - Cache frame counts to avoid repeated disk access
 - Add option for temporal stride
 - Consider raising warning instead of silent padding
-

Lines 313-322: Get Item Method

```
def __getitem__(self, idx):
    folder, label = self.samples[idx]
    fps = self._load_frames(folder)
    frames = []
    for p in fps:
        img = Image.open(p).convert("RGB")
        img = self.transform(img) if self.transform is not None else transforms.ToTensor()(img)
        frames.append(img)
    video = torch.stack(frames, dim=1) # (C, T, H, W)
    return video, label
```

What they do:

- Gets folder path and label
- Loads frame paths (handled by `_load_frames`)
- Opens each image, converts to RGB
- Applies transform if provided
- Stacks frames along dimension 1 (time)
- Returns (C,T,H,W) tensor and label

Improvements:

- Add error handling: Try-except around `Image.open` for corrupt files
 - Use list comprehension: `frames = [self.transform(Image.open(p).convert("RGB")) for p in fps]`
 - Add caching for faster repeated access
 - Validate tensor shape before returning
 - Consider lazy loading for memory efficiency
-

Lines 324-325: Test Dataset

```
ds = FolderSequenceDataset(DATA_ROOT / "train", transform=frame_transform)
print("train samples:", len(ds), "classes:", ds.classes)
```

What they do:

- Creates dataset instance for testing
- Prints sample count and class names

Improvements:

- Print more info: Samples per class, example sample shape
 - Test loading one sample: `video, label = ds[0]`
 - Verify output format
-

Lines 327-328: Model Definition Comment

```
# model definition
class R3D18_Reg(nn.Module):
```

What they do:

- Comment indicating model section
- Defines R3D-18 based model

Improvements:

- Better comment: Explain R3D-18 architecture
 - Add reference to paper
-

Lines 329-331: Model Init

```
def __init__(self, num_classes=3, pretrained=False, dropout_p=0.5, freeze_backbone=False):
    super().__init__()
    self.backbone = torchvision.models.video.r3d_18(pretrained=pretrained)
```

What they do:

- Initializes with configurable parameters
- Loads R3D-18 backbone (pretrained if specified)
- R3D-18 is 3D ResNet with 18 layers

Improvements:

- Add type hints
- Check torchvision version: R3D-18 requires recent version
- Add option for other backbones: R3D-50, MC3-18, etc.
- Print whether using pretrained weights

Lines 333-335: Modify Backbone

```
in_f = self.backbone.fc.in_features if hasattr(self.backbone.fc, 'in_features') else 512
self.backbone.fc = nn.Identity()
```

What they do:

- Gets input features of final layer (512 for R3D-18)
- Replaces final FC layer with Identity (pass-through)
- This removes classification head, keeping only features

Improvements:

- Add assertion: `assert in_f > 0, "Invalid feature dimension"`
- Log: `print(f"Backbone feature dimension: {in_f}")`
- More robust: `in_f = self.backbone.fc.in_features` (no need for `hasattr` check)

Lines 337-339: Freeze Backbone Option

```
if freeze_backbone:
    for p in self.backbone.parameters():
        p.requires_grad = False
```

What they do:

- If `freeze_backbone=True`, disables gradient computation for backbone
- Used for feature extraction only (faster, less memory)

Improvements:

- Log frozen parameters: `print(f"Frozen {sum(p.numel() for p in self.backbone.parameters())} parameters")`
- Consider partial freezing: Freeze early layers, fine-tune later layers
- Add method to unfreeze later: `def unfreeze_backbone(self)`

Lines 341-347: Custom Head

```
self.dropout = nn.Dropout(dropout_p)
self.fc1 = nn.Linear(in_f, 256)
self.bn_fc = nn.BatchNorm1d(256)
self.fc2 = nn.Linear(256, num_classes)

nn.init.kaiming_normal_(self.fc1.weight, nonlinearity='relu')
nn.init.xavier_normal_(self.fc2.weight)
```

What they do:

- Dropout layer (default 0.5 = 50% dropout)
- First linear: 512 → 256
- BatchNorm for 256 features
- Second linear: 256 → num_classes (3)
- Kaiming init for fc1 (optimized for ReLU)
- Xavier init for fc2 (good for final layer)

Improvements:

- Add bias initialization: `nn.init.constant_(self.fc1.bias, 0)`
- Consider deeper head: Add more layers for complex datasets
- Add residual connection from features to output
- Initialize BatchNorm weights: `nn.init.constant_(self.bn_fc.weight, 1)`

Lines 349-355: Forward Pass

```
def forward(self, x):
    feats = self.backbone(x)
    x = self.dropout(feats)
    x = F.relu(self.bn_fc(self.fc1(x)))
    x = self.dropout(x)
    x = self.fc2(x)
    return x
```

What they do:

- Passes input through backbone to get features (512-dim)
- Applies dropout
- `fc1` → BatchNorm → ReLU (256-dim)
- Applies dropout again
- `fc2` to get logits (3-dim)
- Returns raw logits

Improvements:

- Return features too for visualization: `return x, feats`
- Add shape assertions for debugging
- Consider feature normalization before classifier
- Add option to return probabilities: `return F.softmax(x, dim=1)`

Lines 357-358: Instantiate Model

```
model = R3D18_Reg(num_classes=NUM_CLASSES, pretrained=PRETRAINED, dropout_p=0.5, freeze_backbone=FREEZE_BACKBONE).to(DEVICE)
print(model)
```

What they do:

- Creates model instance with config parameters
- Moves to device (GPU/CPU)
- Prints model architecture

Improvements:

- Count parameters: `total_params = sum(p.numel() for p in model.parameters())`
- Count trainable: `trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)`
- Use torchsummary: `from torchsummary import summary; summary(model, (3, 32, 128, 128))`
- Test forward pass before training

Lines 360-366: Create Dataloaders

```
train_ds = FolderSequenceDataset(DATA_ROOT / "train", transform=frame_transform)
val_ds   = FolderSequenceDataset(DATA_ROOT / "val", transform=val_transform, classes=train_ds.classes)
test_ds  = FolderSequenceDataset(DATA_ROOT / "test", transform=val_transform, classes=train_ds.classes)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS, pin_memory=True)
val_loader   = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=max(1, NUM_WORKERS//2), pin_memory=True)
test_loader  = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=max(1, NUM_WORKERS//2), pin_memory=True)
```

What they do:

- Creates three datasets sharing same class ordering
- Creates DataLoaders with:
 - `pin_memory=True`: Faster CPU→GPU transfer
 - Fewer workers for val/test (at least 1)
 - Shuffle only for training

Improvements:

- Add drop_last=True for training: Avoid small last batch
 - Add prefetch_factor: prefetch_factor=2 for better pipelining
 - Add persistent_workers=True: Faster epoch transitions
 - Add timeout: timeout=60 to handle stuck workers
 - Consider weighted sampler for imbalanced datasets
-

Lines 368-373: Compute Class Weights

```
counts = np.zeros(NUM_CLASSES, dtype=int)
for _, lbls in train_loader:
    for l in lbls.numpy():
        counts[l] += 1
total = counts.sum()
class_weights = torch.tensor(total / (NUM_CLASSES * counts.clip(min=1)), dtype=torch.float32).to(DEVICE)
```

What they do:

- Initializes counter for each class
- Iterates through entire training set counting labels
- Computes balanced weights: total/(num_classes × count)
- clip(min=1) prevents division by zero
- Converts to tensor and moves to device

Improvements:

- More efficient: Count from dataset directly without loader iteration
 - Add smoothing: counts + 1 to prevent extreme weights
 - Print weights: print(f"Class weights: {class_weights.cpu().numpy()}")
 - Save weights for reproducibility
 - Consider using sklearn: from sklearn.utils.class_weight import compute_class_weight
 - Visualize distribution: Plot histogram of class counts
-

Lines 374: Print Class Info

```
print("Class counts:", counts, "Class weights:", class_weights.cpu().numpy())
```

What they do:

- Prints counts and computed weights for verification

Improvements:

- Format better: Print as table
 - Add percentages: counts/total*100
 - Plot bar chart of distribution
-

Lines 376-377: Loss Function

```
# loss, optimizer, scheduler
criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
```

What they do:

- Creates CrossEntropyLoss with:
 - weight=class_weights: Applies class balancing
 - label_smoothing=0.1: Soft targets (prevents overconfidence)

Improvements:

- Consider focal loss for extreme imbalance
 - Experiment with label_smoothing values: 0.0 to 0.2
 - Add ignore_index if needed for missing classes
-

Lines 379-386: Optimizer with Differential LR

```

if PRETRAINED and not FREEZE_BACKBONE:

    backbone_params = list(model.backbone.parameters())
    head_params = [p for n,p in model.named_parameters() if "backbone" not in n]
    optimizer = torch.optim.AdamW([
        {'params': backbone_params, 'lr': LR*0.2},
        {'params': head_params, 'lr': LR}
    ], weight_decay=WEIGHT_DECAY)

```

What they do:

- If using pretrained and not frozen:
 - Gets backbone parameters separately
 - Gets head parameters (everything else)
 - Creates AdamW with two parameter groups:
 - Backbone: 1/5 of base LR (gentle fine-tuning)
 - Head: Full LR (faster learning)
 - Applies weight decay to both

Improvements:

- Add learning rate warmup for stability
- Consider even more gradual: Backbone at LR*0.1
- Add gradient clipping per group
- Log parameter group info
- Consider discriminative learning rates: Different LR per layer

Lines 387-389: Standard Optimizer

```

else:
    optimizer = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)

```

What they do:

- If training from scratch or frozen: Use single LR for all parameters

Improvements:

- Consider higher LR when training from scratch
- Add momentum parameter: `betas=(0.9, 0.999)`

Lines 391-392: Learning Rate Scheduler

```

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3, verbose=True)
scaler = torch.amp.GradScaler('cuda' if DEVICE.type=='cuda' else 'cpu')

```

What they do:

- ReduceLROnPlateau: Monitors val_loss
 - mode='min': Lower is better
 - factor=0.5: Reduces LR by half
 - patience=3: Wait 3 epochs before reducing
 - verbose=True: Prints when LR changes
- GradScaler for mixed precision

Improvements:

- Consider CosineAnnealingLR for smoother schedule
- Add min_lr: `min_lr=1e-7` to prevent too-small LR
- Consider OneCycleLR for faster convergence
- Add warmup scheduler for first few epochs

Lines 394-397: Training Initialization

```

best_val_acc = 0.0
stale = 0
PATIENCE = 6

for epoch in range(1, NUM_EPOCHS+1):

```

What they do:

- Tracks best validation accuracy
- Counts stale epochs (no improvement)
- Sets patience for early stopping
- Main training loop from epoch 1 to NUM_EPOCHS

Improvements:

- Track more metrics: best_loss, best_epoch
 - Save training history: `history = {'train_loss': [], 'val_loss': [], ...}`
 - Add timestamp tracking
 - Consider warmup epochs before early stopping
-

Lines 398-399: Training Phase Setup

```
# ---- train ----  
model.train()
```

What they do:

- Comment for clarity
- Sets model to training mode (enables dropout, batch norm training)

Improvements:

- Add epoch timer: `start_time = time.time()`
 - Reset metrics explicitly
-

Lines 400-414: Training Loop

```
train_loss = 0.0  
correct = 0  
total = 0  
for videos, labels in tqdm(train_loader, desc=f"Epoch {epoch} Train", leave=False):  
    videos = videos.to(DEVICE, non_blocking=True)  
    labels = labels.to(DEVICE, non_blocking=True)  
    optimizer.zero_grad()  
    with torch.amp.autocast(device_type='cuda' if DEVICE.type=='cuda' else 'cpu'):  
        outputs = model(videos)  
        loss = criterion(outputs, labels)  
    scaler.scale(loss).backward()  
    scaler.step(optimizer)  
    scaler.update()  
  
    train_loss += loss.item() * videos.size(0)
```

What they do:

- Initializes metric accumulators
- Iterates through training data
- `non_blocking=True`: Async GPU transfer
- `zero_grad`: Clears previous gradients
- `autocast`: Mixed precision forward pass
- Scaled backward pass and optimizer step
- Accumulates weighted loss

Improvements:

- Add gradient clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`
 - Check for NaN: `if torch.isnan(loss): print("NaN detected!"); break`
 - Add gradient monitoring: Track gradient norms
 - Consider gradient accumulation for larger effective batch size
 - Add learning rate logging: `current_lr = optimizer.param_groups[0]['lr']`
-

Lines 415-419: Compute Training Metrics

```

    preds = outputs.argmax(dim=1)
    correct += (preds == labels).sum().item()
    total += labels.size(0)

train_loss /= total
train_acc = correct / total

```

What they do:

- Gets predictions (argmax of logits)
- Counts correct predictions
- Computes average loss and accuracy

Improvements:

- Compute per-class accuracy
- Calculate F1 score, precision, recall
- Use torchmetrics for robust metrics

Lines 421-439: Validation Phase

```

# ---- val ----
model.eval()
val_loss = 0.0
v_correct = 0
v_total = 0
with torch.no_grad():
    for videos, labels in tqdm(val_loader, desc=f"Epoch {epoch} Val", leave=False):
        videos = videos.to(DEVICE, non_blocking=True)
        labels = labels.to(DEVICE, non_blocking=True)
        with torch.amp.autocast(device_type='cuda' if DEVICE.type=='cuda' else 'cpu'):
            outputs = model(videos)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * videos.size(0)
            preds = outputs.argmax(dim=1)
            v_correct += (preds == labels).sum().item()
            v_total += labels.size(0)
val_loss /= v_total
val_acc = v_correct / v_total

```

What they do:

- Sets model to eval mode
- Disables gradients (no_grad)
- Similar to training but no backprop
- Computes validation metrics

Improvements:

- Store predictions for confusion matrix
- Add test-time augmentation (TTA)
- Compute confidence scores
- Track per-class metrics
- Consider `@torch.inference_mode()` instead of `no_grad`

Lines 440: Scheduler Step

```

scheduler.step(val_loss)

```

What they do:

- Updates learning rate based on validation loss
- ReduceLROnPlateau reduces LR if loss plateaus

Improvements:

- Log LR changes
- Consider stepping on accuracy instead: Use different scheduler

Lines 442-444: Print Epoch Results

```
print(f"Epoch {epoch}/{NUM_EPOCHS} Train Loss: {train_loss:.4f}, Train Acc: {train_acc*100:.2f}% | Val Loss: {val_loss:.4f}, Val Acc: {val_acc*100:.2f}%")
```

What they do:

- Prints formatted metrics for current epoch

Improvements:

- Add learning rate: LR: {optimizer.param_groups[0]['lr']:.6f}
- Add time per epoch
- Use logging instead of print for better control
- Add colored output for better readability
- Log to file for record keeping

Lines 446-456: Checkpointing and Early Stopping

```
# checkpoint & early stopping
if val_acc > best_val_acc:
    best_val_acc = val_acc
    stale = 0
    torch.save({
        'epoch': epoch,
        'model_state': model.state_dict(),
        'optimizer_state': optimizer.state_dict(),
        'val_acc': val_acc
    }, MODEL_PATH)
    print("Saved best model.")
```

What they do:

- If validation improves:
 - Updates best accuracy
 - Resets stale counter
 - Saves complete checkpoint (model + optimizer)
 - Prints confirmation

Improvements:

- Save scheduler state too: 'scheduler_state': scheduler.state_dict()
- Save training history
- Add model versioning with timestamp
- Save top-k models, not just best
- Add model metadata: hyperparameters, dataset info

Lines 457-461: Early Stopping Logic

```
else:
    stale += 1
    if stale >= PATIENCE:
        print("Early stopping no improvement")
        break
```

What they do:

- If no improvement: Increment stale counter
- If stale >= PATIENCE (6): Stop training early

Improvements:

- Print stale count: print(f"No improvement for {stale} epochs")
- Consider reducing LR before stopping
- Add option to restore best weights before stopping
- Log reason for stopping

Lines 463-464: Training Complete

```
# final test evaluation
print("Training finished. Best val acc:", best_val_acc)
```

What they do:

- Comment indicating end of training
- Prints best validation accuracy achieved

Improvements:

- Load best model and evaluate on test set
- Print training time
- Save final results to file
- Plot training curves
- Generate confusion matrix on test set

Section 5: Model B with Attention

Lines 466-473: Imports and Setup

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.optim import Adam
from torch.optim.lr_scheduler import ReduceLROnPlateau
from tqdm import tqdm
import random
```

What they do:

- Imports necessary modules
- Adds random for custom augmentation

Improvements:

- Remove duplicate imports (already at top)
- Add typing imports
- Add numpy import

Lines 475-476: Device Setup

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Running on:", device)
```

What they do:

- Sets device and prints it

Improvements:

- Print GPU name if available
- Check memory available

Lines 478-479: Channel Attention Class

```
class ChannelAttention3D(nn.Module):
    def __init__(self, in_channels, reduction=16):
```

What they do:

- Defines channel attention module
- reduction=16: Compression ratio for squeeze operation

Improvements:

- Add type hints
- Add docstring explaining SE (Squeeze-Excitation)
- Make reduction configurable based on channel count

Lines 480-487: Attention Architecture

```
super().__init__()
self.avg_pool = nn.AdaptiveAvgPool3d(1)
self.fc = nn.Sequential(
    nn.Linear(in_channels, in_channels // reduction, bias=False),
    nn.ReLU(inplace=True),
    nn.Linear(in_channels // reduction, in_channels, bias=False),
    nn.Sigmoid()
)
```

What they do:

- AdaptiveAvgPool3d(1): Global average pooling to (B, C, 1, 1, 1)
- FC network: $C \rightarrow C/16 \rightarrow C$
- ReLU in middle
- Sigmoid at end (attention weights 0-1)
- bias=False: Common in SE blocks

Improvements:

- Add max pooling branch: Combine avg and max pooling
- Consider using both squeeze and excitation
- Add batch normalization
- Initialize weights properly

Lines 489-493: Attention Forward

```
def forward(self, x):
    b, c, t, h, w = x.size()
    y = self.avg_pool(x).view(b, c)
    y = self.fc(y).view(b, c, 1, 1, 1)
    return x * y.expand_as(x)
```

What they do:

- Gets input shape (B, C, T, H, W)
- Pools to (B, C, 1, 1, 1) and reshapes to (B, C)
- Passes through FC to get attention weights
- Reshapes back to (B, C, 1, 1, 1)
- Multiplies input by attention weights (channel-wise scaling)

Improvements:

- Add residual: `return x + x * y.expand_as(x)`
- Cache shape instead of extracting each time
- Add option to return attention weights for visualization

Lines 495-496: Model B Definition

```
class ModelB(nn.Module):
    def __init__(self, num_classes=3, dropout_p=0.4):
```

What they do:

- Defines custom model with attention
- dropout_p=0.4 (40% dropout)

Improvements:

- Add more parameters: channels, layers
- Add type hints
- Add docstring

Lines 497-504: First Conv Block

```

    super().__init__()
    # Conv Block 1
    self.conv1 = nn.Conv3d(3, 64, kernel_size=3, padding=1)
    self.bn1 = nn.BatchNorm3d(64)
    self.pool1 = nn.MaxPool3d(2)
    self.drop1 = nn.Dropout3d(dropout_p)
    self.ca1 = ChannelAttention3D(64)

```

What they do:

- Conv3d: 3→64 channels, 3x3x3 kernel
- BatchNorm for 64 channels
- MaxPool reduces all dims by 2
- Dropout3d: Spatial dropout (drops entire feature maps)
- Channel attention for 64 channels

Improvements:

- Add stride to conv instead of separate pooling
- Consider depthwise separable convolutions for efficiency
- Add residual connection
- Initialize weights

Lines 506-512: Second Conv Block

```

    # Conv Block 2
    self.conv2 = nn.Conv3d(64, 128, kernel_size=3, padding=1)
    self.bn2 = nn.BatchNorm3d(128)
    self.pool2 = nn.MaxPool3d(2)
    self.drop2 = nn.Dropout3d(dropout_p)
    self.ca2 = ChannelAttention3D(128)

```

What they do:

- Similar to block 1 but 64→128 channels
- Another round of pooling and attention

Improvements:

- Add more blocks for deeper network
- Use progressive dropout (increase dropout_p)
- Add spatial attention in addition to channel attention

Lines 514-516: Classifier

```

    # Classifier
    self.fc1 = nn.Linear(128 * 8 * 32 * 32, 256)
    self.fc2 = nn.Linear(256, num_classes)

```

What they do:

- Hardcoded input size: 128 channels × 8 (T) × 32 (H) × 32 (W)
- This assumes specific input size after pooling
- fc1: Reduces to 256 features
- fc2: Final classification

Improvements:

- CRITICAL: Calculate size dynamically to avoid hardcoding
- Add dropout between fc1 and fc2
- Add batch norm after fc1
- Add more FC layers for capacity
- Use adaptive pooling before FC to handle any input size

Lines 518-524: Model B Forward

```
def forward(self, x):
    x = self.pool1(F.relu(self.bn1(self.conv1(x))))
    x = self.drop1(self.ca1(x))
    x = self.pool2(F.relu(self.bn2(self.conv2(x))))
    x = self.drop2(self.ca2(x))
    x = x.view(x.size(0), -1)
    x = F.relu(self.fc1(x))
    return self.fc2(x)
```

What they do:

- Block 1: Conv→BN→ReLU→Pool→Dropout→Attention
- Block 2: Conv→BN→ReLU→Pool→Dropout→Attention
- Flatten with view
- fc1→ReLU
- fc2 (logits)

Improvements:

- Add dropout before fc2: `x = F.dropout(x, p=0.5, training=self.training)`
- Print shapes for debugging
- Add batch norm before fc2
- Consider returning features for visualization

Lines 526-534: Augmentation Function

```
def augment_3d(x):
    # x: tensor (C, T, H, W)
    if random.random() < 0.5:
        x = x.flip(-1) # horizontal flip
    if random.random() < 0.5:
        x = x.flip(-2) # vertical flip
    if random.random() < 0.5:
        shift = random.randint(-2, 2)
        x = torch.roll(x, shifts=shift, dims=1)
    return x
```

What they do:

- Takes single video tensor (C, T, H, W)
- 50% chance: Horizontal flip (flip last dim)
- 50% chance: Vertical flip (flip second-to-last dim)
- 50% chance: Temporal shift ± 2 frames (rolls time dimension)

Improvements:

- Add rotation: `transforms.functional.rotate(x, angle)` for each frame
- Add scaling/zoom augmentation
- Add Gaussian noise: `x + torch.randn_like(x) * 0.01`
- Consider mixup or cutmix for video
- Make augmentation probabilities configurable
- Apply transformations consistently across all frames in sequence
- Add temporal reversal: `x.flip(1)` to reverse video

Lines 536: Class Weights

```
class_weights = torch.tensor([0.57575756, 1.5405406, 1.6285714], device=device)
```

What they do:

- Hardcoded class weights from previous computation
- Moved directly to device

Improvements:

- Compute from actual data instead of hardcoding
- Add comment explaining where these came from
- Make configurable or load from file
- Verify these match current dataset distribution

Lines 537: Loss Function

```
criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
```

What they do:

- Creates weighted CrossEntropyLoss with label smoothing

Improvements:

- Already well configured!
 - Could experiment with different label_smoothing values
-

Lines 539-541: Model and Optimizer Setup

```
model = ModelB(num_classes=3).to(device)
optimizer = Adam(model.parameters(), lr=1e-4, weight_decay=1e-4)
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3, verbose=True)
```

What they do:

- Creates ModelB instance
- Adam optimizer with L2 regularization
- ReduceLROnPlateau scheduler

Improvements:

- Use AdamW instead: `torch.optim.AdamW(...)`
 - Print model summary
 - Count parameters
 - Test forward pass before training
 - Consider higher LR since training from scratch: `lr=1e-3`
-

Lines 542: GradScaler

```
scaler = torch.amp.GradScaler('cuda')
```

What they do:

- Creates gradient scaler for mixed precision

Improvements:

- Make device-dependent: `'cuda' if device.type=='cuda' else 'cpu'`
 - Add enabled parameter for conditional mixed precision
-

Lines 544-547: Training Setup

```
num_epochs = 30
best_val_acc = 0.0
patience = 5
stale_epochs = 0
```

What they do:

- Sets number of epochs
- Initializes tracking variables
- patience=5 for early stopping

Improvements:

- Track more metrics: best_loss, best_epoch, history
 - Make configurable
 - Add training start time
-

Lines 549: Main Training Loop

```
for epoch in range(num_epochs):
```

What they do:

- Main loop for training

Improvements:

- Use `range(1, num_epochs+1)` for clearer epoch numbering
- Add try-except for graceful interruption

Lines 550-553: Training Phase Start

```
# ----- TRAIN -----  
model.train()  
train_loss, correct, total = 0.0, 0, 0
```

What they do:

- Comment for section
- Sets training mode
- Initializes metric accumulators

Improvements:

- Add epoch timer
- Log epoch start

Lines 555-558: Training Loop with Augmentation

```
for inputs, labels in tqdm(train_loader, desc=f"Training Epoch {epoch+1}", leave=False):  
    inputs, labels = inputs.to(device), labels.to(device)  
    inputs = torch.stack([augment_3d(x) for x in inputs])  
    optimizer.zero_grad()
```

What they do:

- Iterates through training data
- Moves to device
- **Applies augmentation**: Unpacks batch, augments each sample, restacks
- Zeros gradients

Improvements:

- CRITICAL: Augmentation should be on CPU before moving to GPU for efficiency
- Augment BEFORE moving to device:

```
inputs = torch.stack([augment_3d(x) for x in inputs])  
inputs, labels = inputs.to(device), labels.to(device)
```

- Use `non_blocking=True` for async transfer
- Consider using GPU augmentation libraries like Kornia
- Cache augmented samples if dataset is small

Lines 560-563: Forward and Backward Pass

```
with torch.amp.autocast('cuda'):  
    outputs = model(inputs)  
    loss = criterion(outputs, labels)  
  
scaler.scale(loss).backward()  
scaler.step(optimizer)  
scaler.update()
```

What they do:

- Mixed precision forward pass
- Computes loss

- Scaled backward pass
- Optimizer step with gradient scaling

Improvements:

- Add gradient clipping after backward: `scaler.unscale_(optimizer); torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`
 - Check for NaN loss
 - Log gradient norms occasionally
-

Lines 565-568: Accumulate Metrics

```
train_loss += loss.item() * inputs.size(0)
_, preds = torch.max(outputs, 1)
correct += (preds == labels).sum().item()
total += labels.size(0)
```

What they do:

- Accumulates weighted loss
- Gets predictions
- Counts correct predictions
- Counts total samples

Improvements:

- Detach outputs before argmax if not needed: `preds = outputs.detach().argmax(1)`
 - Store per-class metrics
 - Calculate running accuracy for progress bar
-

Lines 570-571: Compute Training Metrics

```
train_loss /= total
train_acc = correct / total
```

What they do:

- Computes average loss and accuracy

Improvements:

- Add F1 score, precision, recall
 - Compute per-class accuracy
-

Lines 573-575: Validation Setup

```
# ----- VALIDATION -----
model.eval()
val_loss, val_correct, val_total = 0.0, 0, 0
```

What they do:

- Sets eval mode (disables dropout, batch norm uses running stats)
- Initializes metric counters

Improvements:

- Add `@torch.inference_mode()` decorator for entire val section
-

Lines 576-584: Validation Loop

```

with torch.no_grad(), torch.amp.autocast('cuda'):
    for inputs, labels in val_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        val_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1)
        val_correct += (preds == labels).sum().item()
        val_total += labels.size(0)

```

What they do:

- Disables gradients and uses mixed precision
- Similar to training loop without backprop or augmentation
- Accumulates validation metrics

Improvements:

- Add progress bar: `tqdm(val_loader, desc=f"Validation Epoch {epoch+1}")`
- Store predictions for confusion matrix
- Add `non_blocking=True`
- Consider test-time augmentation (TTA): Average predictions across multiple augmented versions

Lines 586-587: Compute Val Metrics and Scheduler

```

val_loss /= val_total
val_acc = val_correct / val_total
scheduler.step(val_loss)

```

What they do:

- Computes average validation metrics
- Updates learning rate based on `val_loss`

Improvements:

- Log learning rate after scheduler step
- Consider stepping on accuracy instead

Lines 589-591: Print Epoch Results

```

print(f"\nEpoch [{epoch+1}/{num_epochs}] "
      f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc*100:.2f}% | "
      f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc*100:.2f}%")

```

What they do:

- Prints formatted epoch results

Improvements:

- Add current learning rate
- Add time per epoch
- Color code improvements (green) vs degradation (red)
- Log to file
- Add overfitting indicator: gap between train and val acc

Lines 593-595: Early Stopping Check

```

# Early stopping
if val_acc > best_val_acc:
    best_val_acc = val_acc
    stale_epochs = 0

```

What they do:

- If validation improves: Update best and reset counter

Improvements:

- Add tolerance: Only update if improvement > threshold (e.g., 0.001)
- Track improvement amount

Lines 596-597: Save Best Model

```
torch.save(model.state_dict(), "best_modelB.pth")
print(f"Saved best model (Val Acc: {best_val_acc*100:.2f}%)")
```

What they do:

- Saves model state dict
- Prints confirmation

Improvements:

- Save complete checkpoint:

```
torch.save({
    'epoch': epoch+1,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict(),
    'best_val_acc': best_val_acc,
    'train_loss': train_loss,
    'val_loss': val_loss
}, 'best_modelB.pth')
```

- Add timestamp to filename
- Keep multiple checkpoints (top-k)
- Save to separate directory

Lines 598-601: Early Stopping Trigger

```
else:
    stale_epochs += 1
    if stale_epochs >= patience:
        print("Early stopping triggered")
        break
```

What they do:

- Increments stale counter if no improvement
- Stops training if patience exceeded

Improvements:

- Print current stale count: `print(f"No improvement for {stale_epochs}/{patience} epochs")`
- Load best model before breaking
- Save final model even if not best
- Log final statistics

Summary of Major Improvements Needed

1. Code Organization

- Consolidate duplicate imports at top
- Use config files (YAML/JSON) for hyperparameters
- Create separate functions for train/val/test
- Add type hints throughout
- Better comments and docstrings

2. Data Pipeline

- Compute normalization stats from actual dataset
- More robust frame handling (corrupt images, missing files)
- Dynamic frame sampling strategies
- Better augmentation (mixup, cutmix, stronger transforms)
- Efficient augmentation (on CPU before GPU transfer)

3. Model Architecture

- Dynamic feature size calculation (avoid hardcoding)
- Proper weight initialization
- More regularization (dropout in more places)
- Consider residual connections
- Add option to return intermediate features

4. Training Process

- Gradient clipping to prevent exploding gradients
- Learning rate warmup for stability
- Better learning rate schedules (OneCycleLR, CosineAnnealing)
- Gradient accumulation for effective larger batches
- NaN detection and handling

5. Monitoring and Logging

- Use tensorboard or wandb for visualization
- Per-class metrics (precision, recall, F1)
- Confusion matrices
- Learning rate logging
- Gradient norm monitoring
- Training time tracking

6. Checkpointing

- Save optimizer and scheduler states
- Save training history
- Model versioning with timestamps
- Keep top-k models not just best
- Metadata (hyperparameters, dataset info)

7. Evaluation

- Test set evaluation after training
- Confidence calibration
- Test-time augmentation (TTA)
- Error analysis (which samples fail)
- Attention visualization

8. Code Quality

- Error handling (try-except blocks)
- Input validation (assert statements)
- Logging instead of print statements
- Reproducibility (complete seed setting)
- Memory efficiency (del unused variables)

9. Performance

- Mixed precision validation
- Prefetch factor in dataloaders
- Persistent workers
- Efficient augmentation
- Gradient checkpointing for larger models

10. Experimentation

- Experiment tracking (MLflow, wandb)
- Hyperparameter search
- Cross-validation
- Ensemble methods
- Different architectures comparison

Critical Issues to Fix Immediately

1. **ModelB: Hardcoded flatten size** - Will break with different input sizes
 - Solution: Use AdaptiveAvgPool3d before FC layers
2. **Augmentation on GPU** - Inefficient, should be on CPU
 - Solution: Augment before .to(device)
3. **Missing gradient clipping** - Risk of exploding gradients

- Solution: Add clip_grad_norm_ with max_norm=1.0

4. **No test set evaluation** - Can't assess final performance

- Solution: Load best model and evaluate on test set

5. **Hardcoded class weights** - May not match current dataset

- Solution: Compute from actual training data

6. **Duplicate imports** - Poor code organization

- Solution: Consolidate all imports at top

7. **No normalization** - Models train better with normalized inputs

- Solution: Add Normalize transform with dataset statistics

8. **Missing error handling** - Code will crash on corrupt data

- Solution: Add try-except around image loading

9. **Inconsistent device handling** - Mix of strings and torch.device

- Solution: Use torch.device consistently

10. **No training resumption** - Can't continue interrupted training

- Solution: Save and load complete checkpoints with epoch info